P12 Parallax

CRE132 worksheet

What you'll make

A layered background that gives the level depth. As you run, the sky stays still, the far clouds drift slowly and the near clouds move a bit faster, but all of them move slower than the level. Things far away seem to move less: that's **parallax**.

You'll build the three background layers from tiled sprites, then make them move with one short script. You'll also learn why the *order* in which scripts update matters for anything that follows the camera.

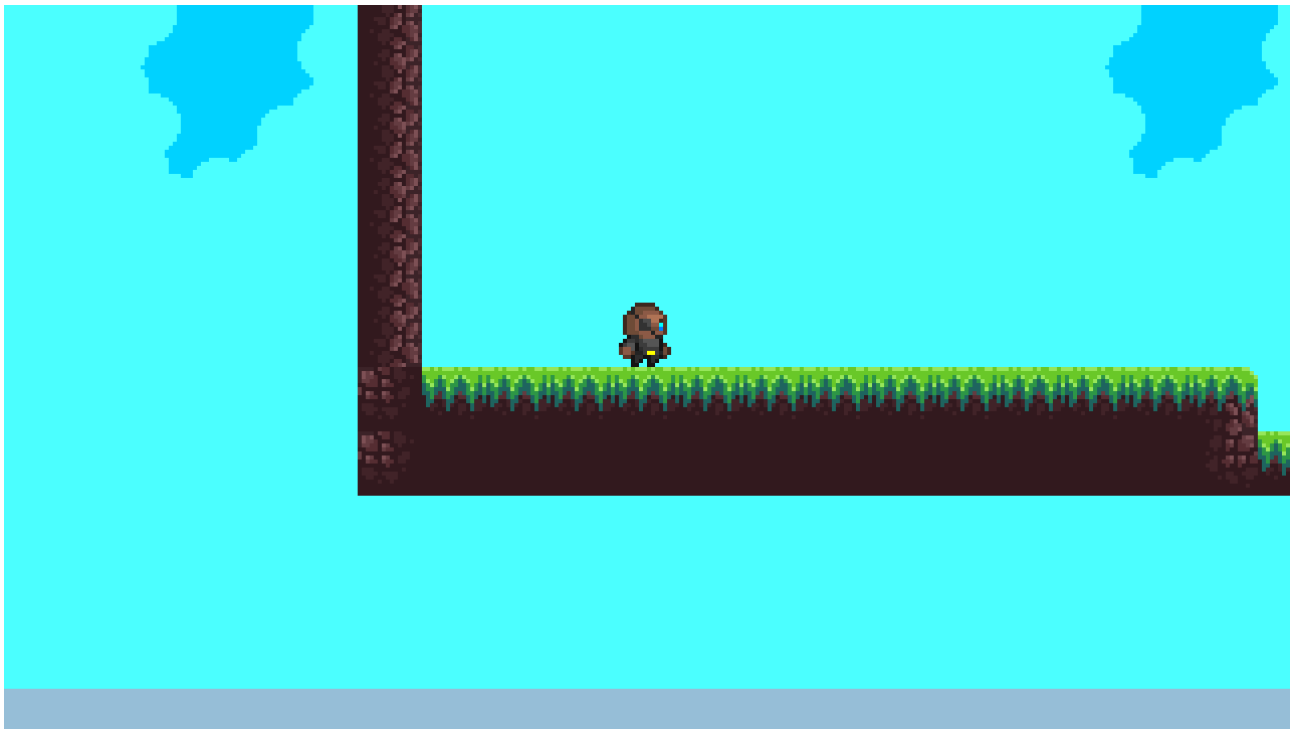


Figure 1: The level with its layered sky and clouds behind it.

Before you start

1. Unzip `CRE132_P12_Parallax_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_P12_Parallax_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Platformer/P12_Parallax`.
3. Press **Play** and run (A/D or arrows) and jump (Space) along the level. There's no background yet: behind the level is just the camera's plain background colour. Press Play again to stop.

The three background pictures are in `Assets/_game/Art/Backgrounds/ElvGames`: `PS_Tileset_07_Background00` (the sky), `01` (far clouds) and `02` (near clouds). They're already imported as sprites with **Wrap Mode: Repeat** and **Mesh Type: Full Rect**, which tiling needs.

> **Making a script:** in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. The background layers

1. Right-click an empty part of the Hierarchy and choose **Create Empty**. Name it `Backgrounds` and check its **Position** is 0, 0, 0. It's only a folder to keep the layers together.
2. Drag `PS_Tileset_07_Background00` from the Project window onto **Backgrounds** in the Hierarchy. It becomes a child with a **Sprite Renderer**. Rename it `Sky (far)`.
3. Do the same with `PS_Tileset_07_Background01`, renamed `Far Clouds (middle)`, and `PS_Tileset_07_Background02`, renamed `Near Clouds (near)`.
4. Select all three children (click the first, Shift-click the last). In the **Sprite Renderer**, set **Draw Mode** to **Tiled** and **Size** to **186** × **15**. Under **Additional Settings**, set **Sorting Layer** to **Background**.
5. Select each child on its own and set its **Position** and **Order in Layer**:

Object	Position X	Position Y	Order in Layer
Sky (far)	31	7.5	−30
Far Clouds (middle)	31	6	−20
Near Clouds (near)	31	4.5	−10

- **Sorting Layer Background**, with **Order in Layer** −30, -20 and −10, puts the sky at the back and the near clouds in front of it. The tilemap is at 0, so the level draws in front of all three.
- **Tiled** repeats the picture to fill the size instead of stretching it. A width of 186 is about three times the level, and X 31 is the middle of the level, so the layers never run out as they move.

Test it: press Play and run along the level. The sky and clouds are behind the level, but they slide past at exactly the same speed as the ground, so they look painted on a flat wall. The script fixes that.

2. The ParallaxLayer script

Make a folder `Assets/_game/Scripts/P12_Parallax` and create `ParallaxLayer` in it:

```
// Moves a background layer along with the camera by a set amount, so far-away layers seem
// to move slowly and near ones quickly. Follow Amount 1 = stays still on screen (the
// sky), 0 = moves with the level.
using UnityEngine;

// Run after everything else, once the camera has moved this frame, or the layers would
// wobble.
[DefaultExecutionOrder(1000)]
public class ParallaxLayer : MonoBehaviour
{
    [SerializeField] private float followAmount = 0.5f;

    // Close to 1, so the layer moves almost with the camera up and down: hills don't
    // bob when you jump.
    [SerializeField] private float verticalFollowAmount = 0.9f;

    private Transform cameraTransform;
    private Vector3 startPosition;
    private Vector3 cameraStartPosition;

    private void Start()
    {
        cameraTransform = Camera.main.transform;
        startPosition = transform.position;
        cameraStartPosition = cameraTransform.position;
```

```
    }

    private void LateUpdate()
    {
        // How far the camera has moved since the start, and the layer moves a share of
        // that.
        Vector3 cameraMovement = cameraTransform.position - cameraStartPosition;
        float x = startPosition.x + cameraMovement.x * followAmount;
        float y = startPosition.y + cameraMovement.y * verticalFollowAmount;
        transform.position = new Vector3(x, y, startPosition.z);
    }
}
```

What's going on:

- **`Start` remembers two positions**: where the layer started and where the camera started. `Camera.main` finds the camera tagged **MainCamera**.
- **Each frame** the layer moves by the camera's movement times `followAmount`. With **1** it moves exactly with the camera, so it looks fixed on screen, like a sky far away. With **0** it doesn't follow at all, so it moves with the level. Values in between give the depth.
- **`verticalFollowAmount`** is separate and set higher than the sideways amount, so the background moves less on screen when you jump and doesn't bob up and down.
- **`[DefaultExecutionOrder(1000)]`** makes this script run after the others. Cinemachine moves the camera during the frame; if the layers moved before it, they'd be a frame behind and wobble.

Add it: select all three background sprites and **Add Component > Parallax Layer**. Then set each one:

Object	Follow Amount	Vertical Follow Amount
Sky (far)	1	1
Far Clouds (middle)	0.8	0.95
Near Clouds (near)	0.5	0.85

Test it: press Play and run. The sky stays still, the far clouds drift slowly and the near clouds move faster.

3. Tuning it in Play mode

Press Play, select **Near Clouds (near)** and drag across the **Follow Amount** label to change the number while you run. Try 0 (the clouds stick to the level), 1 (they stick to the screen) and values in between.

Changes made in Play mode are lost when you stop. If you find values you like, write them down and type them in again afterwards.

Check your work

- ☐ **Backgrounds** has three children, each **Tiled** at 186 × 15 on the **Background** sorting layer, with orders −30, −20 and −10.
- ☐ The sky doesn't move on screen while you run.
- ☐ The far clouds move slower than the near clouds, and both move slower than the level.
- ☐ Jumping doesn't make the background bob up and down much.
- ☐ The layers move smoothly with no wobble, and never run out at the edges.
- ☐ Compare with `CRE132_P12_Parallax_Finished` if something behaves differently.

Key points

- **Parallax is just "move less than the camera".** The further away a layer is, the more it follows the camera.
- **Tiled sprites** are an easy way to get repeating backgrounds.
- **The order of updates matters** for anything that follows the camera. `LateUpdate` and `DefaultExecutionOrder` make sure the camera has moved first.

If something goes wrong

- **NullReferenceException in Start:** `Camera.main` found nothing. The **Main Camera** must keep its **MainCamera** tag.
- **A layer moves too much, or strangely:** Parallax Layer has also been added to the **Backgrounds** parent, so the parent and the sprite both move. Remove it from the parent.
- **The background jitters while you run:** the `[DefaultExecutionOrder(1000)]` line is missing, or the code is in `Update` instead of `LateUpdate`.
- **A layer is one small picture, or stretched:** its **Draw Mode** isn't **Tiled**, or its **Size** is still the picture's own size. Set **Tiled** and **186** × **15**.
- **A layer runs out and you see the plain camera background:** its **Size** is less than 186 wide, or its **Position X** isn't
  31.
- **The clouds cover the level:** check the **Sorting Layer** is **Background** and the **Order in Layer** is below 0.
- **The sky hides the clouds:** the **Order in Layer** values are the wrong way round. The sky needs the lowest number, −30.

Try it

1. Add a foreground layer in front of the level, with a **Follow Amount** below 0 (for example −0.3), so bushes pass in front of the player faster than the level.
2. Make the clouds drift on their own as well, by adding a little to `startPosition.x` each frame.
3. Instead of a very wide sprite, move the layer along by its own width when the camera gets past it, so a narrow sprite wraps round for ever.